Chapter 0

Prerequisites and Notation

0.1 ★ Notation

Excerpted from [1], transcribed using GPT then lightly edited by me.

This section is a reference for notation that we will use throughout the course. It may be updated with notation used throughout the lecture notes. You are not responsible for memorizing all the notation for the Week 1 Quiz, except to the extent that it is used in other material below.

- $:=$ refers to deterministic assignment
- If S is a set, then $x \twoheadleftarrow S$ denotes that x is chosen uniformly from S
- If A is a randomized algorithm, then $y \leftarrow A(x)$ denotes running A on input x with a uniform random tape and assigning the output to y.
- $\wedge$ denotes Boolean conjunction (the AND operator)
- $\vee$ denotes Boolean disjunction (the OR operator)
- $\exists$ denotes the existential quantifier and $\forall$ denotes the universal quantifier. For example: "for every even integer x, there exists an integer k such that $x = 2k$" can be written as "$\forall x \in \mathbb{Z}$ even, $\exists k \in \mathbb{Z}$, $x = 2k$."
- $\oplus$ denotes the exclusive-or (XOR) operator; this operator can be applied to single bits or entire strings (in the latter case, the XOR is bitwise)
- $\{0,1\}^n$ is the set of all bit-strings of length n
- $\{0,1\}^{\leq n}$ is the set of all bit-strings of length at most n
- $\{0,1\}^*$ is the set of all finite bit-strings; $\{0,1\}^+$ is the set of all non-empty, finite bit-strings
- 0^n (resp., 1^n) denotes the string comprised of n zeroes (resp., n ones)
- $\in$ denotes set membership
- $\cup$ denotes the set union
- $\cap$ denotes set intersection
- $\|x\|$ denotes the length of the binary representation of the (positive) integer x, written with leading bit 1. Note that $\log x < \|x\| \leq \log x + 1$

- $|x|$ denotes the length of the binary string x (which may have leading 0s) or the absolute value of a real number x. $|S|$ denotes the size of the set S.
- $[x]$ denotes the set $\{1, 2, \ldots, x\}$ for a positive integer x.
- $O(\cdot)$ is used for asymptotic running times
- `0x` denotes that digits are being represented in hexadecimal. `0b` denotes that digits are being represented in binary.
- $x \| y$ and (x, y) are used interchangeably to denote concatenation of the strings x and y
- $\Pr[X]$ denotes the probability of event X
- $\log x$ denotes the base-2 logarithm of x

Cryptographic notation

- n is the security parameter
- PPT stands for "probabilistic polynomial time"
- $\mathcal{A}^{\mathcal{O}(\cdot)}$ denotes the algorithm $\mathcal{A}$ with oracle access to $\mathcal{O}$
- k typically denotes a secret key (as in private-key encryption and MACs)
- (pk, sk) denotes a public/private key pair (for public-key encryption and digital signatures)
- $\perp$ denotes a generic error symbol
- $\mathsf{negl}(n)$ denotes a negligible function; see Definition **??**
- $\mathsf{poly}(n)$ denotes an arbitrary polynomial

Number theory

- $\mathbb{Z}$ denotes the set of integers: $\mathbb{Z} = \{\ldots, -2, -1, 0, 1, 2, \ldots\}$
- $\mathbb{N}$ denotes the set of natural numbers: $\mathbb{N} = \{0, 1, 2, \ldots\}$
- $[a \bmod m]$ denotes the remainder of a when divided by m
- $x_1 \equiv x_2 \bmod m$ and $x_1 \equiv_m x_2$ both denote congruence modulo m: $[x_1 \bmod m] = [x_2 \bmod m]$.
  Note: $x \equiv y \bmod m$ means that x and y are congruent modulo m, whereas $x = [y \bmod m]$ means that x is equal to the remainder of y when divided by m

0.2 Basic Probability

Excerpted from [1], transcribed using GPT then lightly edited by me.

If E is an event, then $\overline{E}$ denotes the complement of that event; i.e., $\overline{E}$ is the event that E does not occur. By definition, $\Pr[E] = 1 - \Pr[\overline{E}]$. If E_1 and E_2 are events, then $E_1 \wedge E_2$ denotes their conjunction; i.e., $E_1 \wedge E_2$ is the event that *both* E_1 and E_2 occur. By definition, $\Pr[E_1 \wedge E_2] \leq \Pr[E_1]$. Events E_1 and E_2 are said to be *independent* if $\Pr[E_1 \wedge E_2] = \Pr[E_1] \cdot \Pr[E_2]$.

If E_1 and E_2 are events, then $E_1 \vee E_2$ denotes the disjunction of E_1 and E_2; that is, $E_1 \vee E_2$ is the event that *either* E_1 *or* E_2 occurs. It follows from the definition that $\Pr[E_1 \vee E_2] \geq \Pr[E_1]$. The *union bound* is often a very useful upper bound of this quantity.

Proposition 1 (Union Bound)**.** *For any events E_1 and E_2,*

$$\Pr[E_1 \vee E_2] \leq \Pr[E_1] + \Pr[E_2].$$

For any events $E_1, \ldots, E_k$,

$$\Pr\left[\bigvee_{i=1}^{k} E_i\right] \leq \sum_{i=1}^{k} \Pr[E_i].$$

The *conditional probability* of E_1 given E_2, denoted $\Pr[E_1 \mid E_2]$, is defined as

$$\Pr[E_1 \mid E_2] \stackrel{\text{def}}{=} \frac{\Pr[E_1 \wedge E_2]}{\Pr[E_2]}$$

as long as $\Pr[E_2] \neq 0$. (If $\Pr[E_2] = 0$ then $\Pr[E_1 \mid E_2]$ is undefined.) This represents the probability that event E_1 occurs, given that event E_2 has occurred. It follows immediately from the definition that

$$\Pr[E_1 \wedge E_2] = \Pr[E_1 \mid E_2] \cdot \Pr[E_2];$$

equality holds even if $\Pr[E_2] = 0$ as long as we interpret multiplication by zero on the right-hand side in the obvious way.

We can now easily derive Bayes' theorem.

Theorem 1 (Bayes' Rule)**.** *If $\Pr[E_2] \neq 0$ then*

$$\Pr[E_1 \mid E_2] = \frac{\Pr[E_2 \mid E_1] \cdot \Pr[E_1]}{\Pr[E_2]}.$$

Proof. This follows because

$$\Pr[E_1 \mid E_2] = \frac{\Pr[E_1 \wedge E_2]}{\Pr[E_2]} = \frac{\Pr[E_2 \wedge E_1]}{\Pr[E_2]} = \frac{\Pr[E_2 \mid E_1] \cdot \Pr[E_1]}{\Pr[E_2]}.$$

□

Proposition 2 (Law of Total Probability)**.** *Let $E_1, \ldots, E_n$ be disjoint events with $\Pr[E_i] > 0$ for all i. Let F be any event.*

$$\begin{aligned}\Pr[F] &\geq \sum_{i=1}^{n} \Pr[F \wedge E_i] \\ &= \sum_{i=1}^{n} \Pr[F \mid E_i] \cdot \Pr[E_i],\end{aligned}$$

with equality when $\Pr[E_1 \vee \cdots \vee E_n] = 1$.

A special case is when we take E_1 and $\overline{E}_1$ as our disjoint events. Taking $F = E_1 \vee E_2$ for any event E_2, we get a potentially tighter version of the union bound:

$$\begin{aligned}\Pr[E_1 \vee E_2] &= \Pr[E_1 \vee E_2 \mid E_1] \cdot \Pr[E_1] + \Pr[E_1 \vee E_2 \mid \overline{E}_1] \cdot \Pr[\overline{E}_1] \\ &\leq \Pr[E_1] + \Pr[E_2 \mid \overline{E}_1].\end{aligned}$$

0.3 Exercises

Exercise 1. Compute the following quantities for $n \geq 11$.

(a) $|\{0,1\}^n|$

(b) $|\{0,1\}^n \cup \{0,1\}^{10}|$

(c) $|\{0,1\}^n \cap \{0,1\}^{10}|$

Exercise 2. Compute the following probabilities for $n \geq 4$ even and $x \twoheadleftarrow \{0,1\}^n$ sampled uniformly at random. Let A be the event that x is a palindrome. Let B be the event that x starts with 00.

(a) $\Pr[x = 0^n]$

(b) $\Pr[A]$

(c) $\Pr[B]$

(d) $\Pr\left[\ \overline{B}\ \right]$

(e) $\Pr[A \text{ AND } B]$

(f) $\Pr[A \text{ OR } B]$

Exercise 3. Suppose X and Y are distributed as follows. The random choice used to define Y is independent of X:

$$X = \begin{cases} 0 & \text{with probability } 2/3 \\ 1 & \text{with probability } 1/3 \end{cases}$$

$$Y = \begin{cases} X & \text{with probability } 3/4 \\ 1 & \text{with probability } 1/4 \end{cases}$$

(a) Compute $\Pr[Y = 1]$. *Hint: Use Law of Total Probability.*

(b) Compute $\Pr[X = 1 \mid Y = 1]$.

Exercise 4. Let $x = 1011000$ and $y = 0110101$.

(a) Compute $x \oplus y$.

(b) Compute $(x \oplus x) \oplus (y \oplus y) \oplus (x \oplus y) \oplus (y \oplus x)$. *Hint: Simplify the expression.*

(c) Suppose z is sampled uniformly at random from $\{0,1\}^7$. What is $\Pr[z = x \oplus y]$?

Exercise 5. Compute the following.

(a) $[17 + 20 \bmod 4]$

(b) $[17 \cdot 82 \bmod 4]$. *Hint: Simplify before multiplying.*

Exercise 6. The following claim is false: "For all positive integers n, either n is composite or there exists a positive integer k such that $n = 2k + 1$."

(a) Negate the claim. *Hint: Apply De Morgan's Laws.*[1]

[1] `https://en.wikipedia.org/wiki/De_Morgan%27s_laws#Extension_to_predicate_and_modal_logic`

(b) Disprove the claim.

Exercise 7. Let $x \in \{0,1\}^n$ with $n \geq 1$. Give a tight asymptotic bound in terms of n for the worst-case running time of each algorithm below. Count each loop test, assignment, arithmetic operation, and print as one step. In part (c), interpret x as an unsigned n-bit integer.

(a) $\mathcal{A}_1(x)$:

```
for i = 1, ..., n:
  print i
```

(b) $\mathcal{A}_2(x)$:

```
for i = 1, ..., n:
  for j = 1, ..., n:
    print j
```

(c) $\mathcal{A}_3(x)$:

```
while x ≠ 0^n:
  x := x − 1
```

(d) $\mathcal{A}_4(x)$:

```
i := 1
while i ≤ n:
  i := 2i
```

Bibliography

[1] Jonathan Katz and Yehuda Lindell. *Introduction to Modern Cryptography.* Chapman and Hall/CRC, 3rd edition, 2021.